

1. Descripción de la solución

Se desarrolló un sistema de facturación utilizando programación orientada a objetos. La aplicación permite registrar clientes, registrar productos con precio y stock, seleccionar un cliente, agregar productos a una factura indicando cantidades, calcular el total y descontar del inventario las cantidades vendidas.

2. Clases utilizadas

- Cliente: almacena nombre, correo y teléfono y valida sus datos.
- Producto: almacena nombre, precio unitario y stock y valida sus valores.
- Factura: administra el cliente, los detalles, el total y el stock.
- DetalleFactura: representa un producto y la cantidad incluida en la factura.

3. Código de Factura.cs

```
using System.Collections.Generic;
using System.Linq;

namespace Ejercicio5.Models
{
    public class Factura
    {
        public Cliente Cliente { get; set; }
        public List<DetalleFactura> Detalles { get; } =
            new List<DetalleFactura>();

        public Factura() { }

        public void AgregarProducto(Producto producto, int cantidad)
        {
            var existente =
                Detalles.FirstOrDefault(d => d.Producto == producto);

            if (existente != null)
                existente.Cantidad += cantidad;
            else
                Detalles.Add(new DetalleFactura
                {
                    Producto = producto,
                    Cantidad = cantidad
                });
        }

        public decimal CalcularTotal()
        {
            return Detalles.Sum(
                d => d.Producto.PrecioUnitario * d.Cantidad);
        }

        public void AplicarFactura()
        {
            foreach (var d in Detalles)
```

```

        d.Producto.Stock -= d.Cantidad;
    }
}

public class DetalleFactura
{
    public Producto Producto { get; set; }
    public int Cantidad { get; set; }

    public override string ToString()
    {
        return $"{Producto.Nombre} x{Cantidad} = " +
            $"{Producto.PrecioUnitario * Cantidad:C}";
    }
}
}

```

Factura mantiene una lista de DetalleFactura. AgregarProducto() agrega un producto o aumenta su cantidad si ya existe. CalcularTotal() suma precio por cantidad y AplicarFactura() descuenta las cantidades del stock.

4. Código de Producto.cs

```
using System;

namespace Ejercicio5.Models
{
    public class Producto
    {
        public string Nombre { get; set; }
        public decimal PrecioUnitario { get; set; }
        public int Stock { get; set; }

        public Producto() { }

        public Producto(string nombre, decimal precioUnitario, int stock)
        {
            Nombre = nombre;
            PrecioUnitario = precioUnitario;
            Stock = stock;
        }

        public bool EsValido(out string error)
        {
            if (string.IsNullOrEmpty(Nombre))
            {
                error = "El nombre del producto es obligatorio.";
                return false;
            }

            if (PrecioUnitario < 0)
            {
                error = "El precio no puede ser negativo.";
                return false;
            }

            if (Stock < 0)
            {
                error = "El stock no puede ser negativo.";
                return false;
            }

            error = null;
            return true;
        }

        public override string ToString()
        {
            return $"{Nombre} - {PrecioUnitario:C} (Stock: {Stock})";
        }
    }
}
```

5. Código de Cliente.cs

```
using System;
using System.Text.RegularExpressions;

namespace Ejercicio5.Models
{

```

```

public class Cliente
{
    public string Nombre { get; set; }
    public string Correo { get; set; }
    public string Telefono { get; set; }

    public Cliente() { }

    public Cliente(string nombre, string correo, string telefono)
    {
        Nombre = nombre;
        Correo = correo;
        Telefono = telefono;
    }

    public bool EsValido(out string error)
    {
        if (string.IsNullOrEmpty(Nombre))
        {
            error = "El nombre es obligatorio.";
            return false;
        }

        if (string.IsNullOrEmpty(Correo) ||
            !Regex.IsMatch(Correo, @"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
        {
            error = "Correo no válido.";
            return false;
        }

        if (string.IsNullOrEmpty(Telefono) ||
            !Regex.IsMatch(Telefono, @"^[0-9\-\/\s]{6,20}$"))
        {
            error = "Teléfono no válido.";
            return false;
        }

        error = null;
        return true;
    }
}

```

6. Lógica de resolución

Registrar cliente: Se obtienen los datos, se crea un Cliente y se ejecuta EsValido(). Si es válido, se agrega a la lista clientes.

Registrar producto: Se obtienen nombre, precio y stock, se crea un Producto y se valida antes de agregarlo a productos.

Seleccionar cliente y producto: Los objetos registrados aparecen en los ComboBox y el usuario selecciona los necesarios.

Agregar a factura: Se valida cliente, producto, cantidad y stock. Después se agrega el producto a la factura.

Calcular total: Se multiplica el precio unitario por la cantidad de cada detalle y se suman los resultados.

Crear factura: Se valida que exista cliente y productos, se descuenta el stock y se muestra el total.

Actualizar interfaz: Los métodos RefreshClientes(), RefreshProductos() y RefreshFacturaDisplay() actualizan los controles.

7. Código de Form1.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;
using Ejercicio5.Models;

namespace Ejercicio5
{
    public partial class Form1 : Form
    {
        private List<Cliente> clientes = new List<Cliente>();
        private List<Producto> productos = new List<Producto>();
        private Factura factura = new Factura();

        public Form1()
        {
            InitializeComponent();
            RefreshClientes();
            RefreshProductos();
            RefreshFacturaDisplay();
        }

        private void btnAgregarCliente_Click(object sender, EventArgs e)
        {
            var nombre = txtClienteNombre.Text.Trim();
            var correo = txtClienteCorreo.Text.Trim();
            var telefono = txtClienteTelefono.Text.Trim();
            var c = new Cliente(nombre, correo, telefono);

            if (!c.EsValido(out string error))
            {
                MessageBox.Show(error, "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return;
            }
        }
    }
}
```

```

        clientes.Add(c);
        RefreshClientes();
        MessageBox.Show("Cliente agregado.", "Info");
        txtClienteNombre.Clear();
        txtClienteCorreo.Clear();
        txtClienteTelefono.Clear();
    }

    private void btnAgregarProducto_Click(object sender, EventArgs e)
    {
        var nombre = txtProductoNombre.Text.Trim();
        var precio = numPrecio.Value;
        var stock = (int)numStock.Value;
        var p = new Producto(nombre, precio, stock);

        if (!p.EsValido(out string error))
        {
            MessageBox.Show(error, "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        productos.Add(p);
        RefreshProductos();
        MessageBox.Show("Producto agregado.", "Info");
        txtProductoNombre.Clear();
        numPrecio.Value = 0;
        numStock.Value = 0;
    }

    private void btnAgregarAFactura_Click(object sender, EventArgs e)
    {
        if (cmbProductos.SelectedItem == null)
        {
            MessageBox.Show("Seleccione un producto.");
            return;
        }

        var producto = (Producto)cmbProductos.SelectedItem;
        var cantidad = (int)numCantidad.Value;

        if (cantidad <= 0)
        {
            MessageBox.Show("Cantidad debe ser mayor que cero.");
            return;
        }

        if (cantidad > producto.Stock)
        {
            MessageBox.Show("Cantidad mayor al stock disponible.");
            return;
        }

        if (cmbClientes.SelectedItem == null)
        {
            MessageBox.Show("Seleccione un cliente para la factura.");
            return;
        }

        factura.Cliente = (Cliente)cmbClientes.SelectedItem;
        factura.AgregarProducto(producto, cantidad);
        RefreshFacturaDisplay();
    }

    private void btnCrearFactura_Click(object sender, EventArgs e)
    {
        if (factura.Cliente == null)
        {

```

```

        MessageBox.Show("Debe seleccionar un cliente.");
        return;
    }

    if (!factura.Detalles.Any())
    {
        MessageBox.Show("La factura no tiene productos.");
        return;
    }

    factura.AplicarFactura();
    var total = factura.CalcularTotal();

    MessageBox.Show($"Factura creada. Total: {total:C}", "Factura");

    factura = new Factura();
    RefreshFacturaDisplay();
    RefreshProductos();
}

private void RefreshClientes()
{
    cmbClientes.Items.Clear();
    foreach (var c in clientes)
        cmbClientes.Items.Add(c);
    cmbClientes.DisplayMember = "Nombre";
}

private void RefreshProductos()
{
    cmbProductos.Items.Clear();
    foreach (var p in productos)
        cmbProductos.Items.Add(p);
    cmbProductos.DisplayMember = "Nombre";
}

private void RefreshFacturaDisplay()
{
    lstFactura.Items.Clear();
    foreach (var d in factura.Detalles)
        lstFactura.Items.Add(d.ToString());

    lblTotal.Text = $"Total: {factura.CalcularTotal():C}";
}
}
}

```

8. Interfaz gráfica

La primera imagen corresponde al diseño de la aplicación. A la izquierda se registran clientes y productos. A la derecha se selecciona el cliente, producto y cantidad para construir la factura.

The screenshot shows a Windows-style application window titled "Sistema de Facturación". The interface is divided into two main sections: registration on the left and invoice creation on the right.

Registrar Cliente (Left Section):

- Fields: "Nombre", "Correo:", "Teléfono".
- Button: "Agregar".

Registrar Producto (Left Section):

- Fields: "Nombre", "Precio:" (with a value of 0.00 and a spinner), "Stoc:" (with a value of 0 and a spinner).
- Button: "Agregar".

Crear Factura (Right Section):

- Field: "Client" (dropdown menu).
- Field: A second dropdown menu.
- Field: A spinner showing the value "1".
- Button: "Agregar".
- Area: A large empty rectangular box for the invoice details.
- Text: "Total: \$0.00".
- Button: "Crear Factura".

9. Controles principales

- txtClienteNombre, txtClienteCorreo y txtClienteTelefono: datos del cliente.
- btnAgregarCliente: registra el cliente.
- txtProductoNombre: nombre del producto.
- numPrecio: precio unitario.
- numStock: stock disponible.
- btnAgregarProducto: registra el producto.
- cmbClientes: selecciona el cliente.
- cmbProductos: selecciona el producto.
- numCantidad: indica la cantidad a facturar.
- btnAgregarAFactura: agrega el producto a la factura.
- lstFactura: muestra los detalles.
- lblTotal: muestra el total.
- btnCrearFactura: finaliza la factura y descuenta el stock.

10. Corrida de la solución

La segunda imagen muestra una ejecución del sistema. Se observa el cliente 'diego' seleccionado, el producto 'Agua' y una cantidad de 1. En la lista aparece 'Agua x1 = \$3.00' y el total mostrado es \$3.00.

The screenshot shows a Windows Forms application titled 'Sistema de Facturación'. It contains two main sections: 'Registrar Cliente' and 'Registrar Producto'. The 'Registrar Cliente' section has three text boxes for 'Nombre', 'Correo', and 'Teléfono', and an 'Agregar' button. The 'Registrar Producto' section has a text box for 'Nombre', a numeric spinner for 'Precio' (set to 0.00), a numeric spinner for 'Stoc' (set to 0), and an 'Agregar' button. To the right, the 'Crear Factura' section has a 'Client' dropdown menu (set to 'diego'), a product dropdown menu (set to 'Agua'), and a quantity spinner (set to 1). There is an 'Agregar' button next to the quantity. Below these, a list box displays 'Agua x1 = \$3.00'. At the bottom of the 'Crear Factura' section, it shows 'Total: \$3.00' and a 'Crear Factura' button.

11. Resultado de la corrida

Elemento	Resultado
Cliente	diego
Producto	Agua
Cantidad	1
Precio unitario	\$3.00
Detalle	Agua x1 = \$3.00
Total	\$3.00

12. Conceptos de programación aplicados

Se aplicaron clases y objetos, encapsulamiento mediante propiedades, constructores, listas genéricas, composición entre Factura y DetalleFactura, validaciones, expresiones regulares, LINQ mediante FirstOrDefault y Sum, valores decimal para precios, eventos y controles de Windows Forms.

13. Conclusión

La solución permite realizar un proceso básico de facturación de manera organizada. Los clientes y productos se almacenan en listas, la factura administra sus detalles, el total se calcula automáticamente y al finalizar se actualiza el stock. La separación de responsabilidades entre las clases facilita el mantenimiento y demuestra el uso de programación orientada a objetos.